

Netflix — High Level Design (v2)

1. Functional Requirements

• User registration and login (email/password, session-based auth)
• Browse movies and TV shows by category, genre, and trending lists
• Search for content by title, genre, or actor
• View detailed information for a movie/show (cast, synopsis, duration, rating)
• Stream video content with adaptive quality
• Resume playback from the last watched position
• Add or remove content from 'My List'
• Create and manage multiple profiles under one account
• Maintain per-profile watch history
• Provide personalized content recommendations
• Download titles for offline viewing on mobile devices

2. Non-Functional Requirements

Quality	Description / Target
High Availability	System should be up and reachable 99.95% of the time (allows ~4.4 hrs downtime/year).
Scalability	Should scale horizontally to support millions of concurrent users during peak hours (e.g., evenings/weekends).
Low Latency	Video should start playing within 1.5 seconds; API responses under 150 ms.
Reliability	Watch progress and user data must not be lost even if a server fails.
Security	Passwords stored as salted hashes; data encrypted in transit (HTTPS/TLS) and at rest.
Durability	Video files and user data must be durably stored (replicated storage, no data loss).
Performance	Smooth streaming with adaptive bitrate even on low/variable network speeds.

3. Back-of-the-Envelope Estimation

Metric	Estimate
Total registered users	250 million
Daily Active Users (DAU)	100 million (~40%)
Peak concurrent streaming users	12 million (~12% of DAU at peak hour)
Average watch time per user/day	~1.8 hours
Average API requests per user/day	~60 (browse, search, progress updates, downloads)
Peak QPS (API)	~69,000 requests/sec
Average video size (per title, all bitrates)	~6 GB

Metric	Estimate
Total content catalog size	18,000 titles x 6 GB ≈ 108 TB (before replication)
Peak network/streaming bandwidth	12M users x 5 Mbps avg stream ≈ 60 Tbps (served via CDN)
Total video storage (with replication)	~325 TB (3x replication)
New content uploaded per day	~75 titles
CDN cache hit ratio (target)	~97%

Updated assumptions vs the original design: ~250M registered users (up from 200M), ~40% daily active, and an offline-download feature adds extra storage and API load.

4. Database Design

USER	PROFILE	MY_LIST	WATCH_HISTORY	CONTENT
user_id (PK)	profile_id (PK)	profile_id (FK, PK)	history_id (PK)	content_id (PK)
email	user_id (FK)	content_id (FK, PK)	profile_id (FK)	title
password_hash	profile_name	added_at	content_id (FK)	description
created_at	created_at		progress	genre
			watched_at	release_year
				duration

SUBSCRIPTION (new)

subscription_id (PK)
user_id (FK)
plan_type
renewal_date

Relationships

- USER (1) -> (N) PROFILE — one account can have many profiles
- PROFILE (1) -> (N) WATCH_HISTORY — one profile has many history entries
- CONTENT (1) -> (N) WATCH_HISTORY — one title appears in many history entries
- PROFILE (N) -> (N) CONTENT — through MY_LIST (many-to-many)
- USER (1) -> (1) SUBSCRIPTION — one account has one active billing plan (new)

5. API Design

Method	Endpoint	Purpose
POST	/api/login	Authenticate user and return session token
GET	/api/content/browse	Get list of content by category/genre
GET	/api/content/search?q=	Search content by title/genre/actor (Elasticsearch-backed)
GET	/api/content/{content_id}	Get details of a specific movie/show
POST	/api/profiles	Create a new profile
GET	/api/profiles/{user_id}	List all profiles under an account
GET	/api/history/{profile_id}	Get watch history for a profile
POST	/api/mylist	Add content to My List
DELETE	/api/mylist/{content_id}	Remove content from My List
PUT	/api/playback/progress	Update playback progress (resume point)
GET	/api/recommendations/{profile_id}	Get personalized recommendations
GET (new)	/api/download/{content_id}	Request an offline-download package for a title (new)

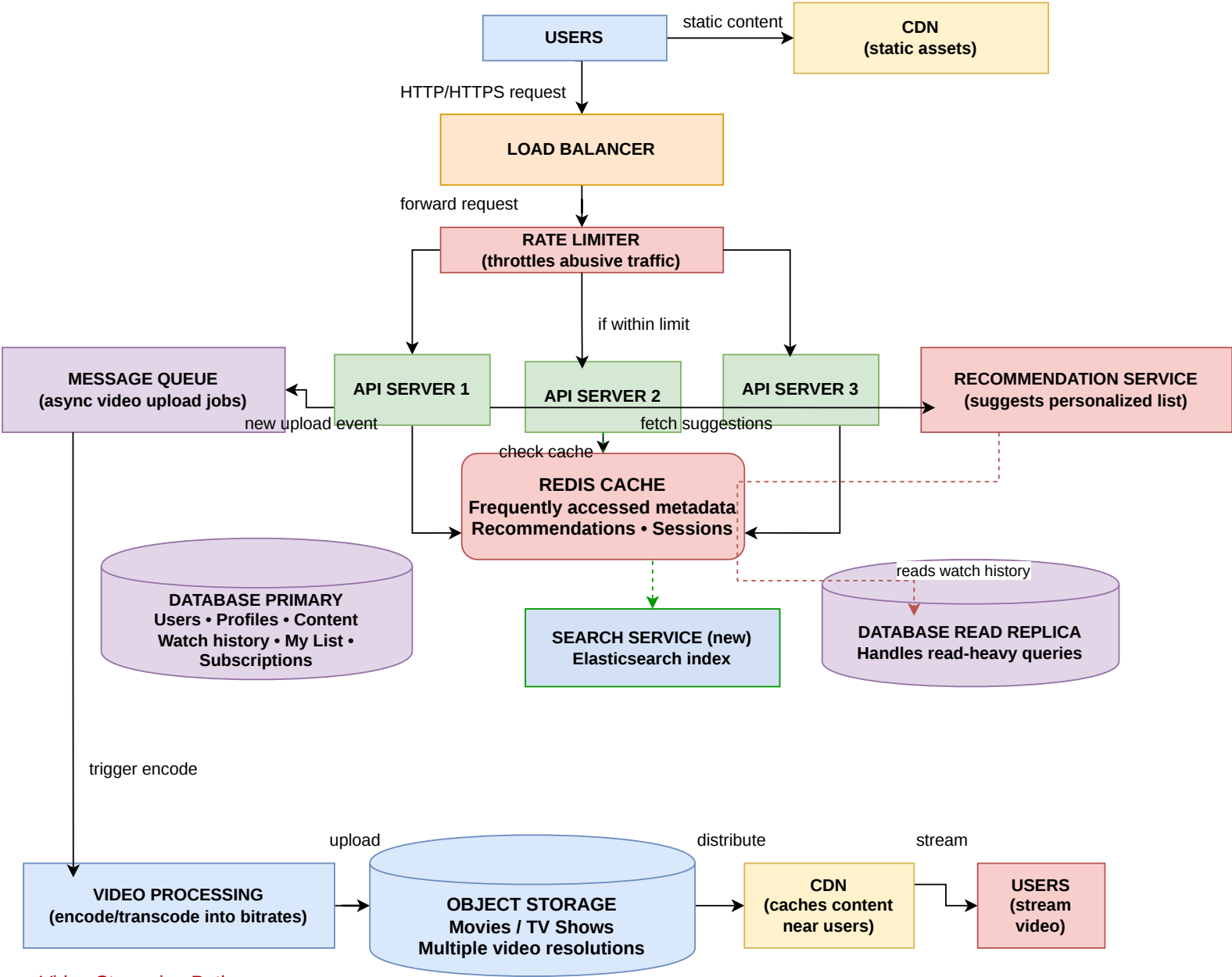
Example: Login API

Request:
POST /api/login
{ "email": "user@example.com",
 "password": "mypassword123" }

Response:
200 OK
{ "user_id": "u123", "token":
 "abc.def.ghi", "message": "Login
 successful" }

6. Architecture Diagram

Flow 1 — User request: **USERS** -> **LOAD BALANCER** -> **RATE LIMITER** -> **API SERVERS** -> **REDIS CACHE / SEARCH SERVICE** -> **DATABASE**.
Flow 2 — Video content: **VIDEO PROCESSING** -> **OBJECT STORAGE** -> **CDN** -> **USERS**. New: dashed green edge shows API servers querying a dedicated Search Service (Elasticsearch) instead of hitting the DB directly for search.



Video Streaming Path